

Chain-of-Memory

Chain-of-Memory: Lightweight Memory Construction with Dynamic Evolution for LLM Agents

<https://arxiv.org/pdf/2601.14287>

记忆系统：记忆库构建→检索记忆→利用检索出的历史记忆
构建做轻，检索不变，利用做重

1.motivation

与其花大量成本来进行记忆库的构建，不如采用轻量级存储，把计算重点放在如何利用检索到的记忆上

对比的基线方法：

Full-Context：不做任何记忆系统，把完整对话历史直接塞进LLM的上下文窗口

RAG (session)：把对话按session为单位切块，每个session作为一个检索单元。粒度较粗，一次检索回来的内容可能包含大量无关信息

<https://arxiv.org/abs/2005.11401>

RAG (turn)：把对话按单轮turn为单位切块，每个turn是一个检索单元。粒度更细，检索更精准，token消耗更少

<https://arxiv.org/abs/2312.10997>

LangMem (LangChain Memory)：先用LLM从对话中提取关键事实/摘要，再对这些提取出的事实做向量检索。

<https://www.langchain.com/blog> 2022年

LangMem 的做法是在构建阶段就用LLM对每段对话提取关键事实：

A-Mem：把对话信息构建成知识图谱，节点之间通过动态笔记链接建立关联。构建阶段需要大量LLM调用来抽取实例和关系

<https://arxiv.org/abs/2502.12110>

Mem0：对对话做增量式事实压缩，同时维护图结构扩展。会持续用LLM压缩和更新记忆

<https://arxiv.org/abs/2504.19413>

第1天："我养了一只猫叫小白"

→ Mem0存储事实： 用户有一只猫，名叫小白

第10天："小白最近生病了，在吃药"

→ 新信息进来，Mem0调用LLM判断：和"小白"相关，是补充信息，不是冲突

→ 合并为： 用户有一只猫叫小白，目前生病在吃药

第20天："小白已经康复了"

→ 新信息进来，Mem0调用LLM判断：和"生病在吃药"冲突，属于状态更新

→ 压缩为： 用户有一只猫叫小白，曾生病现已康复

第30天："我又领养了一只狗叫大黄"

→ 新信息进来，Mem0调用LLM判断：和现有记忆不冲突，是新事实

→ 新增一条： 用户有一只狗，名叫大黄

→ 同时在图结构里建立： 用户 → 拥有宠物 → 小白(猫)、大黄(狗)

A-Mem不做压缩合并。三条关于小白的笔记各自独立存在，通过链接关联。而Mem0会把它们压缩成一条： 用户有一只猫叫小白，曾生病现已康复

A-Mem不维护实体级别的图。它只有笔记之间的相似度链接，没有 用户→养→小白→是→猫 这种结构化的实体-关系图

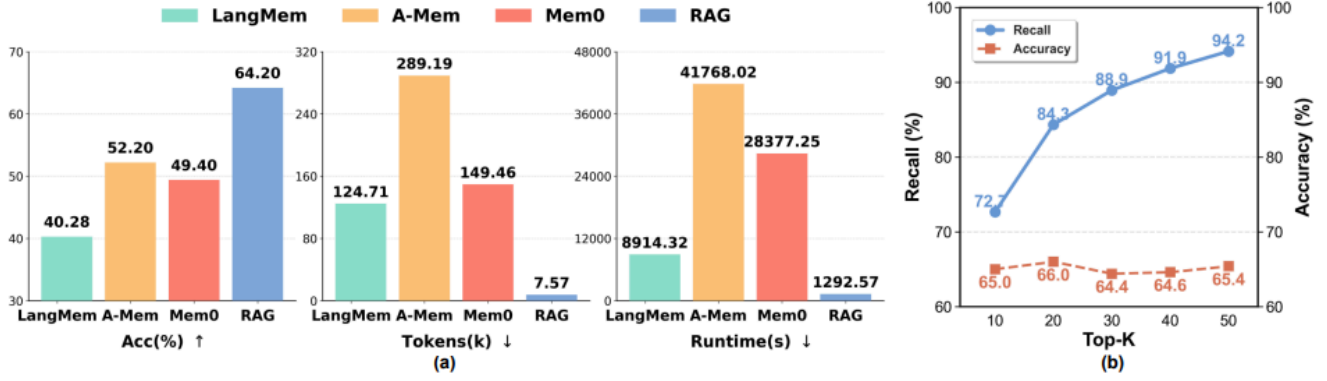


Figure 1: **Empirical limitations of existing paradigms.** (a) Heavy-weight memory construction strategies fail to demonstrate cost-effectiveness. (b) Naive retrieval strategies exhibit a reasoning bottleneck, where retrieved evidence is not effectively utilized for answer generation.

Method	LongMemEval							LoCoMo						
	Accuracy (%) ↑					Cost ↓		Accuracy (%) ↑					Cost ↓	
	S-hop	M-hop	Temp	Kno	Total	Token	Time	S-hop	M-hop	Temp	Kno	Total	Token	Time
Backbone: GPT-4o-mini														
Full-Context	68.59	39.85	44.36	76.92	55.80	112.7	8154	88.59	53.55	50.78	50.00	71.88	31.8	4933
RAG (session)	75.64	54.89	44.36	57.69	59.00	26.6	1512	78.83	52.02	47.16	44.79	65.32	7.5	2601
RAG (turn)	79.49	54.14	48.87	76.92	64.20	7.6	1293	77.76	41.13	44.79	60.44	65.39	1.4	2304
LangMem	49.35	38.35	33.83	35.89	40.20	124.7	8914	66.46	36.17	41.74	38.54	54.02	286.8	3318
A-Mem	73.72	42.86	34.59	55.13	52.20	289.2	41768	74.31	37.58	44.54	37.50	59.09	357.0	8787
Mem0	41.67	45.11	54.14	64.10	49.40	149.5	28377	73.72	40.07	61.05	47.91	63.31	388.4	6326
CoM	84.62	65.41	65.41	83.33	74.20	8.2	2730	83.59	48.94	73.52	46.88	72.86	4.4	3058
Backbone: Qwen3-32B														
Full-Context	73.72	41.35	38.35	70.51	55.20	119.6	11504	86.44	51.06	49.53	45.83	69.74	36.0	2981
RAG (session)	78.21	54.89	47.37	66.67	62.00	27.3	1598	78.83	45.39	44.85	39.58	63.18	7.8	1816
RAG (turn)	82.05	59.40	48.87	74.36	66.00	7.9	2891	75.26	34.04	57.32	41.66	61.88	1.7	2604
LangMem	51.28	39.10	35.33	37.17	41.60	150.2	7635	67.89	38.65	42.05	37.50	55.25	336.6	3850
A-Mem	78.20	39.10	30.08	66.67	53.20	331.8	32949	77.88	41.48	40.49	36.45	60.84	409.7	9721
Mem0	54.49	45.86	43.61	62.82	50.60	156.0	19721	75.74	45.74	53.27	42.70	63.51	456.0	8739
CoM	86.54	69.92	69.17	79.49	76.40	8.8	2002	81.45	46.09	71.96	48.95	70.97	4.8	2050

Table 2: Main results on the LongMemEval and LoCoMo benchmarks. We report Accuracy (%) across four sub-tasks: **S-hop** (Single-hop), **M-hop** (Multi-hop), **Temp** (Temporal), and **Kno** (Knowledge). **Total** denotes the accuracy calculated across the entire dataset (all samples). **Token** refers to the total end-to-end token consumption (in thousands, k), and **Time** indicates the total runtime in seconds (s). Best results are highlighted in bold.

二、方法

1.记忆构建与检索

构建

原始对话 $\rightarrow m_{i,j} = (x, \tau, \rho, \mathbf{e})$ (python代码实现)

- x 表示原始文本内容
- τ 表示时间戳
- ρ 表示说话者角色
- $\mathbf{e}$ 表示文本 (x) 的嵌入向量

检索

query与 $\mathbf{e}_i$ 进行余弦相似度计算 $\rightarrow$ 选Top-K条记忆，构成候选库

2.构建记忆链

- 候选库中选前L个作为起始锚点（类比写作文，从k个候选论点中选出最好的L个作为文章论点）
- $S_{\text{gate}}(m) = \cos(\mathbf{m}, \mathbf{q}) \times \cos(\mathbf{m}, \mathbf{C}_z^{(t)})$
其中第一项表示候选库中节点与问题的相关性，第二项表示与候选库中节点与当前记忆链的相关性
两项得分同时高才可作为后继节点接入当前记忆链

3.自适应截断：

扩展不是无限进行的。如果当前步最优候选的门控分数相比上一步出现骤降： $s_{*t} < \beta \times s_{t-1}$ ，则停止构造该记忆链

假设用户有很长的对话历史，现在问："我上次旅行在哪里吃了好吃的拉面？"

第一阶段检索完成后，Top-K=6的候选池是：

- m1: "我去年十月去了东京旅行" (相关度0.85)
- m2: "我在新宿一家店吃了拉面，特别好吃" (相关度0.82)
- m3: "那家店叫一兰拉面" (相关度0.75)
- m4: "我最近在学做意大利面" (相关度0.60)
- m5: "东京的天气很好，去了浅草寺" (相关度0.58)
- m6: "我下周要交项目报告" (相关度0.30)

这6个片段是孤立的，普通RAG会直接全部拼接塞进prompt，CoM接下来要组织它们

初始化：取Top-L=2个作为锚点，启动两条链：

- 链1: [m1: 去年十月去了东京旅行]
 - 链2: [m2: 在新宿吃了拉面很好吃]
-

链1的扩展过程：

第1步，对剩余候选算门控分数（跟query的相关度 × 跟当前链的一致性）：

- m2：跟query相关(高) × 跟"东京旅行"连贯(高) → Sgate = 0.78
 - m3：跟query相关(中) × 跟"东京旅行"连贯(中) → Sgate = 0.52
 - m4：跟query相关(中) × 跟"东京旅行"连贯(低) → Sgate = 0.25
 - m5：跟query相关(低) × 跟"东京旅行"连贯(高) → Sgate = 0.40
 - m6：跟query相关(低) × 跟"东京旅行"连贯(低) → Sgate = 0.08
- m2得分最高，加入链1 → [m1 → m2]

第2步，链的embedding更新了（现在包含东京旅行+新宿拉面），继续评估：

- m3：跟query相关(中) × 跟"东京旅行→新宿拉面"连贯(高) → Sgate = 0.65
 - m5：跟query相关(低) × 跟"东京旅行→新宿拉面"连贯(中) → Sgate = 0.30
 - m4：→ Sgate = 0.18
 - m6：→ Sgate = 0.06
- 上一步分数是0.78，这一步最优是0.65， $0.65 > 0.5 \times 0.78 = 0.39$ ，没触发截断。m3加入 → [m1 → m2 → m3]

第3步，继续评估：

- m5：→ Sgate = 0.22
 - m4：→ Sgate = 0.12
 - m6：→ Sgate = 0.05
- 上一步是0.65，这一步最优是0.22， $0.22 < 0.5 \times 0.65 = 0.325$ ，骤降触发截断，链1停止扩展。

链1最终结果：去了东京 → 新宿吃了拉面很好吃 → 那家店叫一兰

3.实验

数据集：

LongMemEval <https://arxiv.org/abs/2410.10813>

LoCoMo <https://arxiv.org/abs/2402.17753>

- 单跳问题（single-hop）
- 多跳问题（multi-hop）
- 时间推理问题（temporal reasoning）
- 知识问题（knowledge）
- Total：整个数据集全部样本上的总体准确率；
- Token：端到端 token 总消耗量，单位为千（k）
- Time：总运行时间，单位为秒

Method	LongMemEval							LoCoMo						
	Accuracy (%) ↑					Cost ↓		Accuracy (%) ↑					Cost ↓	
	S-hop	M-hop	Temp	Kno	Total	Token	Time	S-hop	M-hop	Temp	Kno	Total	Token	Time
<i>Backbone: GPT-4o-mini</i>														
Full-Context	68.59	39.85	44.36	76.92	55.80	112.7	8154	88.59	53.55	50.78	50.00	71.88	31.8	4933
RAG (session)	75.64	54.89	44.36	57.69	59.00	26.6	1512	78.83	52.02	47.16	44.79	65.32	7.5	2601
RAG (turn)	79.49	54.14	48.87	76.92	64.20	7.6	1293	77.76	41.13	44.79	60.44	65.39	1.4	2304
LangMem	49.35	38.35	33.83	35.89	40.20	124.7	8914	66.46	36.17	41.74	38.54	54.02	286.8	3318
A-Mem	73.72	42.86	34.59	55.13	52.20	289.2	41768	74.31	37.58	44.54	37.50	59.09	357.0	8787
Mem0	41.67	45.11	54.14	64.10	49.40	149.5	28377	73.72	40.07	61.05	47.91	63.31	388.4	6326
CoM	84.62	65.41	65.41	83.33	74.20	8.2	2730	83.59	48.94	73.52	46.88	72.86	4.4	3058
<i>Backbone: Qwen3-32B</i>														
Full-Context	73.72	41.35	38.35	70.51	55.20	119.6	11504	86.44	51.06	49.53	45.83	69.74	36.0	2981
RAG (session)	78.21	54.89	47.37	66.67	62.00	27.3	1598	78.83	45.39	44.85	39.58	63.18	7.8	1816
RAG (turn)	82.05	59.40	48.87	74.36	66.00	7.9	2891	75.26	34.04	57.32	41.66	61.88	1.7	2604
LangMem	51.28	39.10	35.33	37.17	41.60	150.2	7635	67.89	38.65	42.05	37.50	55.25	336.6	3850
A-Mem	78.20	39.10	30.08	66.67	53.20	331.8	32949	77.88	41.48	40.49	36.45	60.84	409.7	9721
Mem0	54.49	45.86	43.61	62.82	50.60	156.0	19721	75.74	45.74	53.27	42.70	63.51	456.0	8739
CoM	86.54	69.92	69.17	79.49	76.40	8.8	2002	81.45	46.09	71.96	48.95	70.97	4.8	2050

Table 2: Main results on the LongMemEval and LoCoMo benchmarks. We report Accuracy (%) across four sub-tasks: **S-hop** (Single-hop), **M-hop** (Multi-hop), **Temp** (Temporal), and **Kno** (Knowledge). **Total** denotes the accuracy calculated across the entire dataset (all samples). **Token** refers to the total end-to-end token consumption (in thousands, k), and **Time** indicates the total runtime in seconds (s). Best results are highlighted in bold.

Accuracy：答案是否正确由LLM判定
具体规则：

- 1. 普通问题
只要模型回答中包含正确答案，就判为 `yes`
- 2. 时间推理
语义等价或者包含必要的中间推理步骤，就判为正确；计算天数出现相差一天的误差也不扣分
- 3. 知识更新
即使模型在给出最新答案的同时提到了一些旧信息，也可以判为正确
- 4. 用户偏好
不一定有唯一的标准答案，而是判断模型回答是否满足数据集提供的评分规则
- 5. 无法回答的问题
如果问题本身无法根据历史信息回答，那么模型正确地拒绝回答也算正确

大模型：
GPT-4omini
Qwen3-32B (NonThinking Mode)

- 性能提升来自CoM的方法设计，而不是模型自己"想"出来的

- 降低token成本

编码器：Qwen3-Embedding-8B

消融实验：

- 去掉整个Com框架
- 去掉构建记忆链（DMCE）
- 去掉自适应截断（APT）
- 使用整个Com框架

超参数分析：

k=20 β =0.5时最佳

上界分析：

Gemini全文输入：完整对话历史 + 问题 → 直接交给Gemini → 生成回答

Oracle设定：人工标注的证据 + 问题 → 直接交给GPT-4o-mini/Qwen3-32B → 生成回答

Method	Sub-category Acc (%)				Total	Tokens	RunTime
	Single-hop	Multi-hop	Temporal	Knowledge	Acc (%)	(k)	(s)
Full Context (Gemini-2.5-pro)	90.38	86.47	90.98	88.46	89.20	122.89	8512.10
Evidence Only (GPT-4o-mini)	83.33	82.71	69.17	88.46	80.20	0.30	668.36
Ours (GPT-4o-mini)	84.62	65.41	65.41	83.33	74.20	8.24	2730.03
Evidence Only (Qwen3-32B)	89.74	81.20	70.68	85.90	81.80	0.35	456.20
Ours (Qwen3-32B)	86.54	69.92	69.17	79.49	76.40	8.81	2002.45

错误分析：

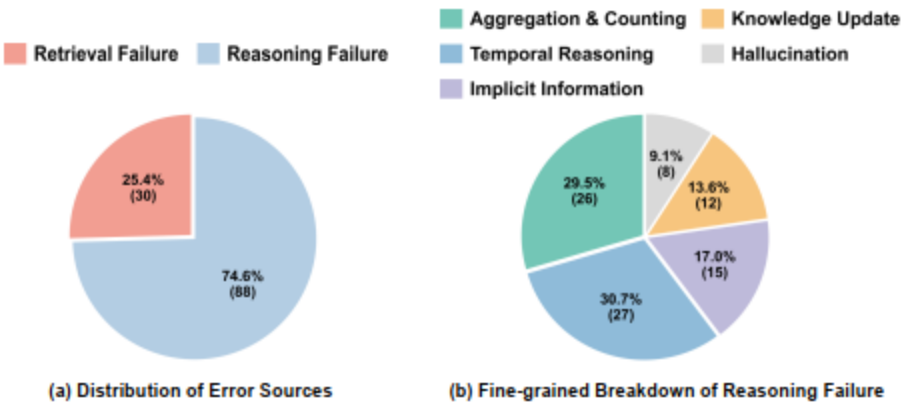


Figure 5: Distribution of error types on the Long-MemEval dataset. Reasoning failures constitute the majority of errors compared to retrieval failure.

推理失败占绝大多数，为 74.6%；检索失败只占相对较小的一部分，为 25.4%

推理失败中：

时间推理：30.7%

聚合与计数：29.5%

- **聚合**：把多个对话片段中的相关事实汇总起来；
- **计数**：按照问题规定的条件，对汇总后的有效事实去重并计算数量。

LongMemEval 中有一道题：

我需从商店取回或退回多少件衣物？

标准答案是 3 相关信息分散在多个会话中，涉及：

- 一件需从干洗店取回的深蓝色西装外套
- 一双需退换的 Zara 靴子
- 一双更换尺码后需取回的新靴子
- 同一双靴子还在后面的对话中被再次提到

1.论文说明 GPT-4o-mini 和 Qwen3-32B 均用于答案生成与 LLM-as-a-Judge 评价，但没有明确说明报告是自评还是GPT与Qwen交叉评判、采样配置和重复评判协议，因此评价过程无法被严格复现

2.人工评判答案准确率和llm评判的答案准确率是否一致没有做实验

3.答案错误的情况主要是推理失败导致的，推理失败中时间推理和聚合与计数推理占据大多数

可以再多读解决时间推理和聚合与计数推理方向的论文

4. 问题：用户现在是否适合参加马拉松？

记忆1：用户报名了马拉松

记忆2：用户三个月前膝盖受伤

记忆3：医生建议半年内避免长距离跑步

关键证据如果没有进入记忆链，要再重新发起一轮检索，找寻到关键信息